

MobileRAG: A Fast, Memory-Efficient, and Energy-Efficient Method for On-Device RAG

Apresentação didática do método e de sua proposta arquitetural.

 **Autores:** Taehwan Park, Geonho Lee, Minsoo Kim
Citação: Park, Lee e Kim, 2025

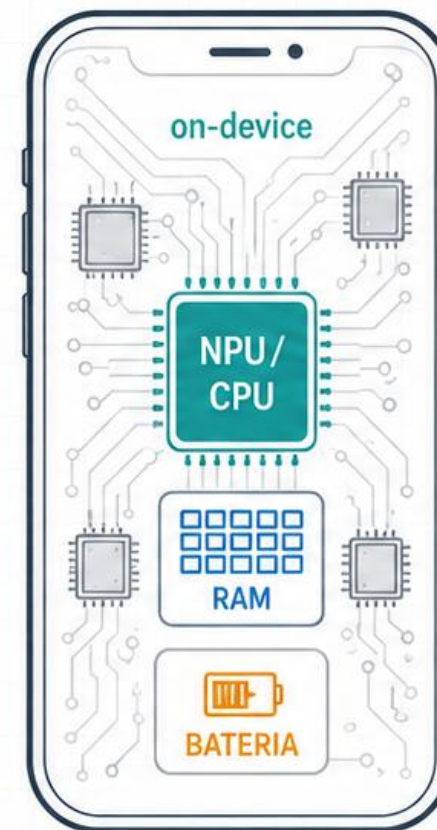
 **Nome:** Fabio Eidi Ara

 **Professor:** Me. André Luiz da Silva

 **Curso:** Tecnologia em Análise e Desenvolvimento de Sistemas

 **Instituição:** Instituto Federal de São Paulo

 Baseado em Park, Lee e Kim, 2025



Privacidade preservada

Dados permanecem no aparelho.



Uso eficiente da memória

Menor consumo de RAM.



Eficiência energética

Menor consumo da bateria.



Funciona offline

Não depende de internet.



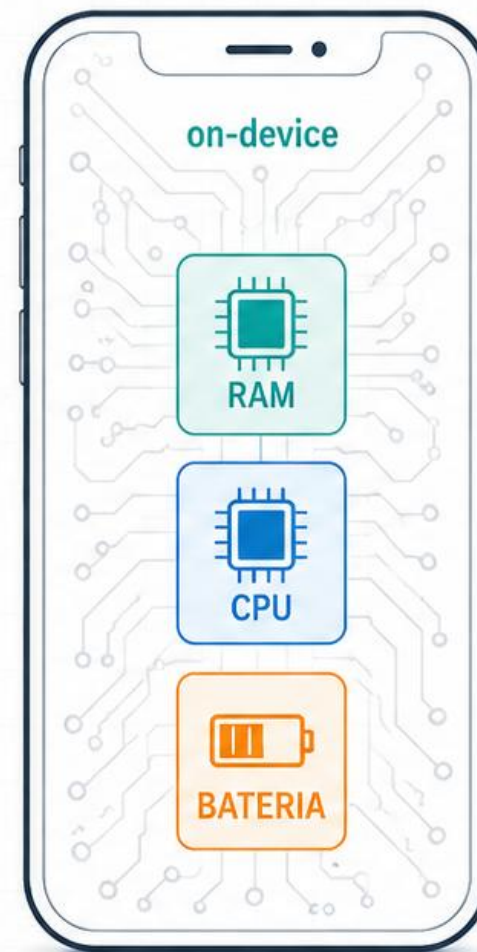
Ideia central: executar RAG no próprio smartphone, com menos memória, menor consumo de energia e menor latência.

Problema acadêmico e utilidade do método

-  **Problema:** executar RAG em smartphones com pouca RAM, CPU limitada e bateria finita.
-  RAG tradicional foi pensado para ambientes com mais recursos computacionais.
-  Em celulares, isso pode aumentar a latência, aquecer o aparelho e gastar mais energia.
-  **Objetivo do artigo:** tornar viável a busca e geração local de respostas em documentos do próprio usuário.
-  **Utilidade prática:** consultar PDFs, anotações e arquivos locais com privacidade e sem internet.

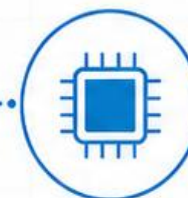


RAG = Retrieval-Augmented Generation = buscar trechos relevantes + gerar uma resposta com base neles.



RAM limitada

Menor capacidade de memória.



CPU limitada

Menor poder de processamento.



Bateria finita

Energia limitada e recarregamentos esporádicos.



Privacidade e uso offline

Dados permanecem no aparelho. Não depende de internet.

Conceitos essenciais do artigo

1

 $\begin{bmatrix} 0.23 \\ -0.11 \\ 0.48 \\ \dots \\ 0.07 \end{bmatrix}$ **Embedding:**

representação numérica de um texto em forma de vetor.

2

**Busca vetorial:**

procura os vetores mais parecidos com a pergunta.

3

**top-k:**

os k resultados mais semelhantes.

4

**sLM:**

small language model, um modelo menor executado no próprio aparelho.

5

**TTFT (Time To First Token):**

tempo até sair a primeira palavra da resposta.

6

**I/O:**

leitura e escrita entre RAM e armazenamento flash.

$$\text{similaridade}(q, d) = \cos(q, d) = \frac{q \cdot d}{||q|| \ ||d||}$$



Essa fórmula apenas mede o quanto a pergunta se parece com um documento.



Comparação com outros métodos

	 Ideia principal	 Uso de RAM	 Carga da CPU	 Uso de disco (I/O)	 Observação
 Naive-RAG	carrega muito contexto e faz muita comparação	alta 	alta 	baixo 	simples, mas pesado para celular
 EdgeRAG	pagina dados para aliviar memória	média 	média-alta 	médio 	melhora a memória, mas ainda envia muito contexto ao modelo
 MobileRAG	combina EcoVector + SCR	baixa 	baixa 	médio 	reduz memória e também reduz o contexto enviado ao sLM



Particularidade do MobileRAG: ele atua em dois pontos ao mesmo tempo — na busca (EcoVector) e na entrada do modelo (SCR).



Pipeline do MobileRAG



$$\text{TTFT} \approx T_{\text{busca}} + T_{\text{redução}} + T_{\text{inferência}}$$

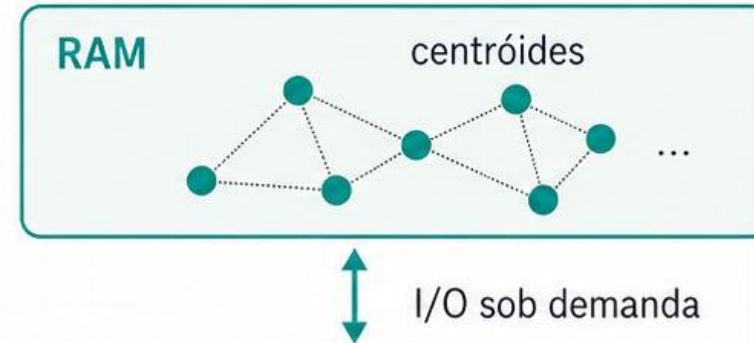
Reduzir busca e quantidade de texto ajuda a resposta a começar mais cedo.



Todo o pipeline é executado **on-device**.

EcoVector: algoritmo da busca

1. Agrupar vetores em clusters.
2. Guardar apenas os centróides na RAM.
3. Guardar as listas invertidas completas no armazenamento flash.
4. Na consulta, comparar a pergunta com os centróides.
5. Carregar do disco apenas o cluster mais promissor.
6. Fazer o top- k dentro desse conjunto menor.



$$c^* = \operatorname{argmax}_j \operatorname{sim}(q, \mu_j)$$

escolhe o centróide mais parecido com a consulta.

Memória RAM $\approx |C| \times d \times 4$ bytes

na RAM fica só a parte compacta do índice.



Ideia-chave: trocar parte do trabalho pesado de RAM por leituras controladas no flash.

SCR: algoritmo de redução de conteúdo



- 1 Divide os documentos em segmentos menores.
- 2 Calcula quão relevante cada segmento é para a pergunta.
- 3 Mantém apenas os segmentos com maior pontuação.
- 4 Envia ao sLM um contexto menor e mais útil.

$$score_i = \cos(e_q, e_i)$$

e_q : embedding da pergunta
 e_i : embedding do segmento i

$$C_{red} = Top-r(score_i)$$

C_{red} : conjunto de segmentos selecionados

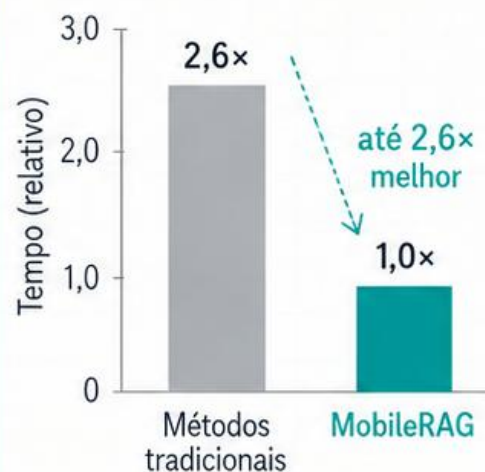
i **token** = pedaço do texto processado pelo modelo.

i Menos tokens -> menos trabalho da CPU -> menor TTFT.

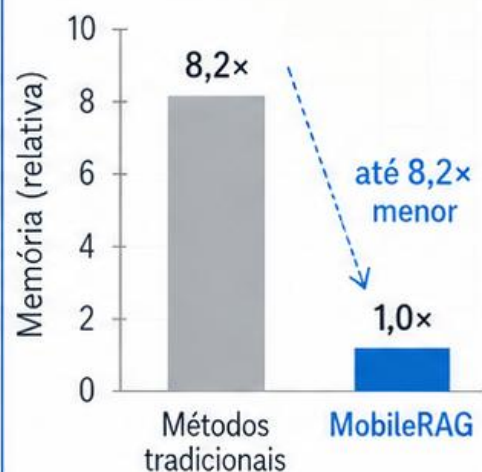
Resultados reportados no artigo

Comparação do MobileRAG com métodos de RAG tradicionais em dispositivos móveis.

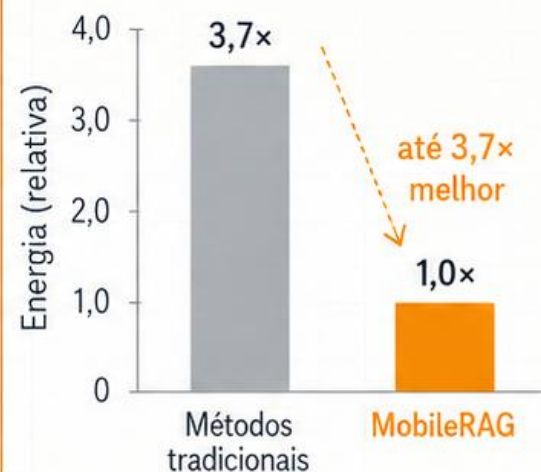
Latência / TTFT até 2,6× melhor



Uso de memória até 8,2× menor



Eficiência energética até 3,7× melhor



Precisão

recall semelhante ou superior em vários cenários

Métodos tradicionais



MobileRAG



Recall semelhante ou superior em diferentes datasets e índices avaliados no artigo.



Os ganhos variam conforme dataset, índice e configuração do dispositivo.



Resumo: MobileRAG melhora desempenho sem perder o foco principal da tarefa de recuperação.

Relação com Organização e Arquitetura de Computadores



Hierarquia de memória: RAM é rápida, mas escassa; flash é maior, porém mais lento.



Trade-off arquitetural: menos CPU em troca de mais I/O controlado.



Energia e calor: muitas operações na CPU aumentam consumo e *thermal throttling*.

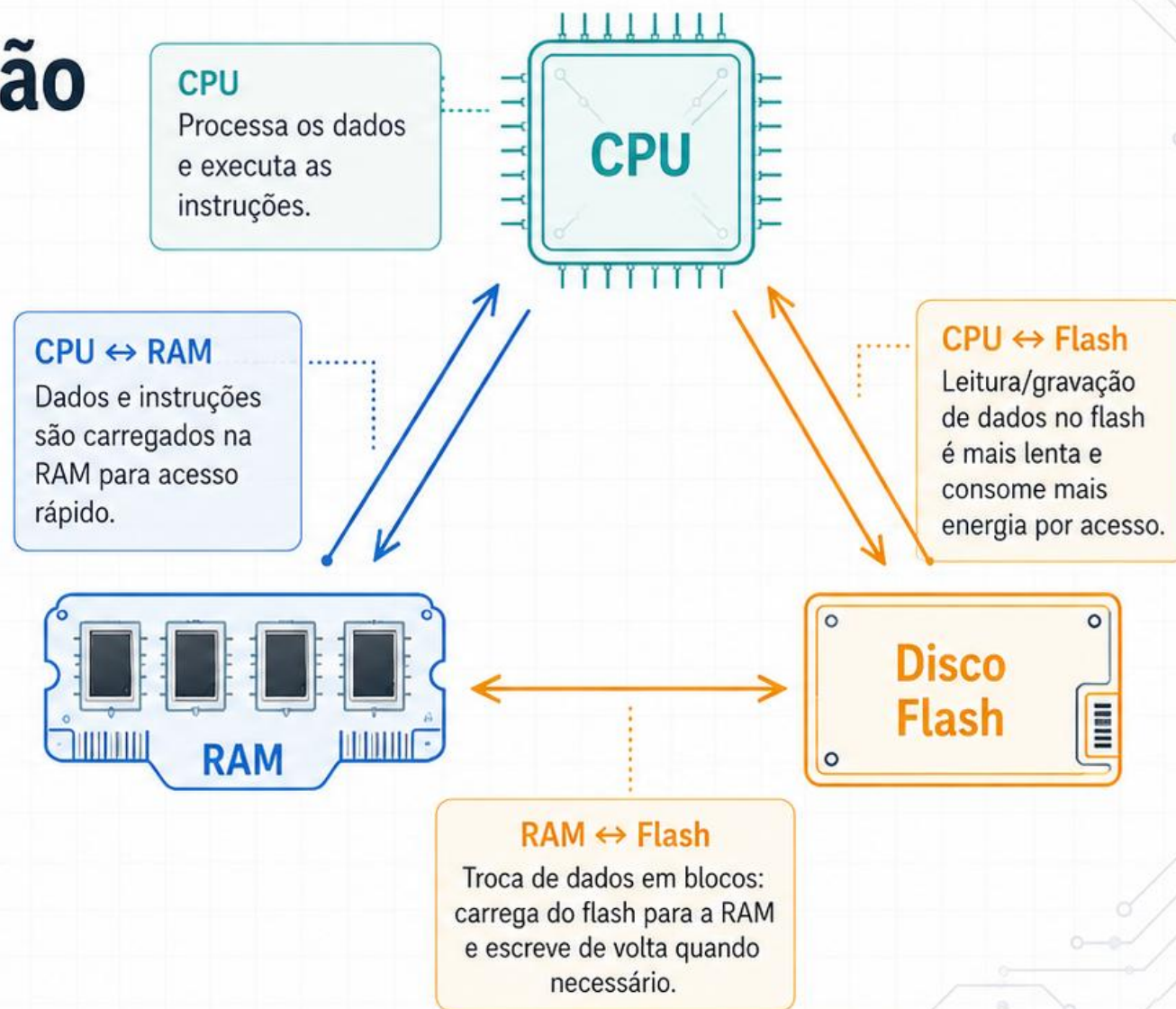


Projeto orientado ao hardware: o software é reorganizado para respeitar os limites físicos do dispositivo.



$$E_{total} \approx E_{CPU} + E_{RAM} + E_{Flash}$$

O MobileRAG busca reduzir principalmente E_{CPU} .



Conclusão



O problema resolvido: tornar o RAG viável em smartphones.



A principal ideia: adaptar o pipeline à arquitetura do hardware móvel.



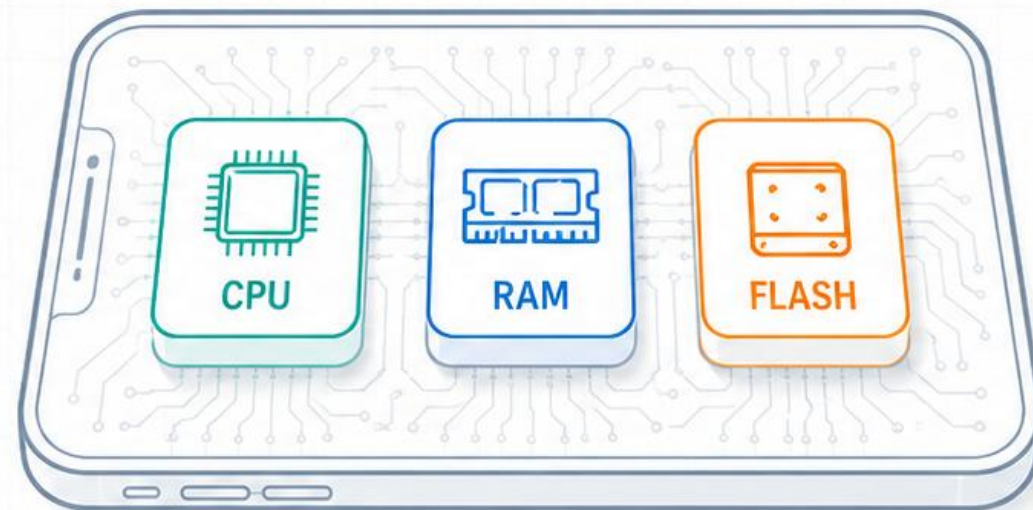
EcoVector reduz pressão sobre a RAM.



SCR reduz o contexto enviado ao sLM e diminui o trabalho da CPU.



O resultado é melhor equilíbrio entre memória, latência, energia e privacidade.



Particularidade do método: MobileRAG não melhora apenas a busca; ele também reduz a entrada do modelo, o que o diferencia de abordagens que atacam só um dos gargalos.



Limite: a qualidade final ainda depende da capacidade do sLM embarcado.